

Análise da Rede de Colaboração de um Repositório *Open Source* por meio de Teoria dos Grafos

Vicenzo Fonseca de Mello Souza¹, Renato Douglas Nascimento Silva de Oliveira¹,
Ana Luiza de Freitas Rodrigues¹,
Kayke Emanuel de Souza Santos¹

¹ Curso de Engenharia de Software, Pontifícia Universidade Católica de
Minas Gerais (PUC Minas)
Belo Horizonte, MG, Brasil
vicenzofms@gmail.com, renatodouglasnso@gmail.com
analuihafreitas12@yahoo.com.br, kaykeeman@gmail.com

Resumo. *Este trabalho apresenta uma ferramenta computacional, desenvolvida do zero (sem bibliotecas prontas de grafos), para modelar e analisar a rede de colaboração de um repositório open source hospedado no GitHub. As interações entre colaboradores, como comentários em issues e pull requests, fechamento de issues, revisões e merges, são mineradas via API do GitHub e modeladas como um grafo simples e direcionado, ponderado pela intensidade de cada tipo de interação. Sobre essa estrutura aplicam-se métricas de redes complexas (centralidade, coesão e detecção de comunidades). O relatório descreve a modelagem, a arquitetura da solução, os algoritmos implementados e os resultados obtidos.*

Abstract. *This work presents a computational tool, built from scratch (without any off-the-shelf graph libraries), to model and analyze the collaboration network of an open source repository hosted on GitHub. Interactions among contributors such as comments on issues and pull requests, issue closing, reviews and merges, are mined through the GitHub API and modeled as a simple, directed graph, weighted by the intensity of each interaction type. Complex-network metrics (centrality, cohesion and community detection) are then applied over this structure. The report describes the modeling, the solution architecture, the implemented algorithms and the obtained results.*

1. Introdução

A colaboração em projetos de *software* livre deixa um rastro rico de interações entre desenvolvedores: pessoas comentam em *issues*, revisam o código umas das outras, aprovam e integram contribuições. Esse rastro pode ser modelado como uma **rede social técnica**, em que os colaboradores são nós e as interações são arestas. A Teoria dos Grafos oferece o instrumental adequado para extrair, dessa rede, quem são os participantes centrais, como se formam grupos de trabalho e quais indivíduos atuam como pontes entre diferentes frentes do projeto.

O objetivo deste trabalho é desenvolver uma ferramenta que (i) mine essas interações de um repositório real do GitHub, (ii) as represente por meio de uma estrutura de grafos implementada do zero, respeitando boas práticas de engenharia de *software*, e (iii) aplique algoritmos e métricas de redes complexas para caracterizar a rede de colaboração. O enunciado proíbe explicitamente o uso de bibliotecas prontas de grafos

(como o `networkX`), de modo que tanto a estrutura de dados quanto todos os algoritmos foram implementados pela equipe.

Este relatório está organizado da seguinte forma. A Seção 2 justifica a escolha do repositório analisado. A Seção 3 descreve a modelagem do grafo e o esquema de pesos. A Seção 4 apresenta a arquitetura da solução e a estrutura de classes. A Seção 5 detalha a estratégia de coleta de dados. A Seção 6 descreve os algoritmos e métricas implementados. A Seção 7 trata da estratégia de testes. A Seção 8 discute os resultados obtidos. A Seção 9 apresenta as telas produzidas pela aplicação. A Seção 10 detalha a divisão de responsabilidades entre os membros e a Seção 11 conclui o trabalho.

2. Escolha do Repositório

O repositório escolhido para a análise foi o `wezterm/wezterm` (<https://github.com/wezterm/wezterm>), projeto *open source* do emulador de terminal WezTerm. A escolha atende ao requisito de popularidade do enunciado: no momento da consulta, o projeto possuía aproximadamente **26.623 estrelas**, **1.500 forks** e **1.474 issues abertas**, além de histórico de desenvolvimento iniciado em fevereiro de 2018. O projeto é majoritariamente escrito em Rust e possui uma comunidade ativa em discussões, *pull requests*, revisões e correções de problemas.

A escolha se justifica por três fatores principais. Primeiro, o volume de interações é suficiente para produzir uma rede social técnica não trivial, com milhares de usuários e arestas. Segundo, o domínio do projeto é tecnicamente rico: por se tratar de um terminal multiplataforma, as discussões envolvem *bugs*, compatibilidade entre sistemas, desempenho, configuração e integração com outros ambientes de desenvolvimento. Terceiro, o projeto possui atividade recente, o que aumenta a chance de encontrar diferentes papéis de colaboração, como mantenedores, revisores recorrentes e colaboradores pontuais.

O *snapshot* local usado no desenvolvimento foi salvo em `src/data/wezterm_wezterm.json`. Esse cache contém **4.462 usuários** e **9.920 interações distintas** já agregadas por origem, destino e tipo de relação. A aplicação exibe a data do cache nos cards de repositório (campo “Cacheado em”) para deixar explícito quando os dados foram minerados.

3. Modelagem do Grafo

3.1. Definição da rede

Cada **usuário** do repositório é representado por um **vértice**; cada **interação** entre dois usuários é representada por uma **aresta direcionada** da origem (quem realizou a ação) para o destino (quem a recebeu). Conforme exigido pelo enunciado, o grafo é **simples** (sem laços nem arestas múltiplas) e **direcionado**; quando uma relação bidirecional precisa ser representada, utilizam-se **arestas anti-paralelas** (uma aresta em cada sentido). Interações de um usuário consigo mesmo (por exemplo, comentar na própria *issue*) são descartadas, pois gerariam laços.

3.2. Tipos de interação e os quatro grafos

A modelagem segue as duas etapas previstas no enunciado. Primeiro, constroem-se **grafos separados**, um para cada natureza de relação; em seguida, um **grafo integrado** consolida todas as interações com pesos diferenciados. A Tabela 1 resume a correspondência entre os grafos e os tipos de interação minerados.

Tabela 1. Os quatro grafos construídos e suas interações.

Grafo	Interações incluídas	Tipos internos
1: Comentários	Comentários em <i>issues</i> e <i>PRs</i>	comentario_issue, comentario_pull_ request
2: Fechamento	Fechamento de <i>issue</i> por outro usuário	fechamento_issue
3: <i>Pull Requests</i>	Revisões e <i>merges</i> de <i>PRs</i>	revisao_pull, merge_pull
Integrado	Combinação ponderada de todas as interações	(todos os anteriores)

O **grafo integrado ponderado** é o objeto principal da análise: nele, o peso de uma aresta $u \rightarrow v$ é a soma dos pesos de todas as interações de u para v , refletindo a intensidade acumulada da colaboração entre os dois.

3.3. Esquema de pesos

Os pesos foram calibrados para refletir a **intensidade de colaboração técnica** de cada interação: ações que exigem maior esforço, comprometimento e envolvimento direto com o código recebem peso maior, conforme orienta o enunciado. A Tabela 2 apresenta o esquema adotado.

Tabela 2. Esquema de pesos por tipo de interação.

Interação	Peso	Natureza
<i>Merge</i> de <i>pull request</i>	5	Integração de código de terceiro ao projeto
Revisão/aprovação de <i>pull request</i>	4	Análise crítica do código alheio
Comentário em <i>issue</i> ou <i>PR</i>	2	Participação na discussão (texto)
Fechamento de <i>issue</i> por outro	1	Ação de triagem, sem texto nem código

A gradação segue uma lógica de **esforço e relevância técnica crescentes**. O *merge* (peso 5) é a interação de maior peso porque representa o ato de integrar a contribuição de outra pessoa ao projeto, envolvendo responsabilidade técnica e comprometimento máximos. A revisão (peso 4) exige leitura cuidadosa e análise crítica do código de terceiros, ficando logo abaixo. O comentário (peso 2) representa participação na discussão: exige elaboração textual, mas não toca diretamente no código.

Justificativa do peso 1 para o fechamento de *issue*: De todas as interações que mapeamos, o fechamento de *issue* é a que exige **menos ação por parte do usuário**. Diferentemente de um comentário, não requer a elaboração de um texto; e, diferentemente de uma revisão ou de um *merge*, não envolve diretamente o código do projeto; trata-se, essencialmente, de uma ação de *triagem* (resolver, organizar ou encerrar uma discussão), frequentemente realizada com um único clique. Ainda assim, o fechamento é uma interação *significativa*: revela um vínculo real entre dois colaboradores (quem agiu sobre a *issue* aberta por outro usuário) e, por isso, não deve ser descartado. Optamos, então, por atribuir-lhe o **menor peso não nulo** (1): a interação é registrada e contribui para a rede, mas com a intensidade compatível com o baixo esforço e o caráter indireto que a caracterizam. Dessa forma, o esquema de pesos preserva uma ordenação coerente,

na qual o sinal de colaboração mais “barato” pesa menos do que aqueles que demandam produção textual (comentário) ou envolvimento direto com o código (revisão e *merge*).

O enunciado permite explicitamente que as regras de ponderação sejam adaptadas pela equipe, desde que interações tecnicamente mais relevantes mantenham peso maior, premissa integralmente respeitada pelo esquema acima.

3.4. Decisões de modelagem: direção e peso por métrica

Algumas métricas são classicamente definidas para grafos não direcionados. Para preservar o rigor, adotamos as seguintes convenções, registradas aqui por transparência:

- **Centralidades de grau, proximidade, intermediação, PageRank e autovetor:** calculadas sobre o **grafo direcionado**, respeitando o sentido das interações.
- **Coefficiente de agrupamento, assortatividade e detecção de comunidades (Louvain):** calculados sobre o **grafo subjacente** (versão não direcionada, em que existe aresta entre u e v se há interação em qualquer sentido), pois são métricas definidas para grafos não direcionados. No caso do Louvain, usa-se o grafo subjacente *ponderado* (somando os pesos das duas direções).
- **Grafos desconexos:** tratados explicitamente. Na proximidade, vértices inalcançáveis recebem distância infinita e aplica-se a normalização de Wasserman–Faust, que pondera pela fração de nós efetivamente alcançáveis, evitando divisão por zero.

4. Arquitetura da Solução

A ferramenta foi organizada em camadas com responsabilidades separadas. O núcleo da solução permanece em Python, enquanto a interface de demonstração é uma aplicação Angular que consome a API HTTP do backend:

```
frontend Angular → backend FastAPI → minerador/cache →  
grafos → analise → frontend/gephi
```

O pacote `minerador` coleta as interações do GitHub e grava um cache JSON; o pacote `backend.fontes` lê esse cache por meio da abstração `FonteDeDados`; o `GrafoServico` transforma os dados minerados nos quatro grafos definidos na modelagem; o `AnaliseServico` orquestra as métricas e serializa os resultados por *username*; e o pacote `gephi` exporta arquivos CSV para visualização externa. Uma regra de projeto central é que os serviços de análise e exportação dependem da abstração `GrafoAbstrato`, podendo operar tanto sobre matriz quanto sobre lista de adjacência.

No backend, a aplicação FastAPI é criada por `criar_app`. Essa função configura CORS, registra tratadores de erro e instancia os serviços compartilhados em `app.state`: `RepositorioServico`, `GrafoServico`, `AnaliseServico`, `FonteCache` e `GerenciadorMineracao`. As rotas são separadas em três grupos: `repositorios` (listagem, exclusão, mineração e status), `grafos` (resumos, visualização e exportação GEPHI) e `analise` (relatório completo e sub-recursos de métricas). O uso de injeção de dependência do FastAPI mantém os serviços testáveis sem acesso real à rede.

O `GerenciadorMineracao` é a implementação única da mineração assíncrona: ele valida pré-condições, exige tokens configurados, registra jobs em

memória, executa o `Minerador` em *background* e atualiza o estado final como concluído ou erro.

O frontend Angular possui três páginas principais: seleção/mineração de repositórios, visão geral dos grafos disponíveis e visualização de métricas. A camada `Api` centraliza as chamadas HTTP, enquanto o serviço `Repositorio` mantém o repositório selecionado, a representação escolhida (matriz ou lista), o tipo de grafo analisado e o acompanhamento de jobs de mineração. Nos cards de repositório, a aplicação também apresenta a data “Cacheado em”, obtida do *mtime* do JSON minerado.

Além dos pacotes-núcleo, a versão atual possui uma aplicação web completa: um *backend* FastAPI, responsável por expor a API HTTP, e um *frontend* Angular, responsável pela demonstração da ferramenta e visualização das métricas. Os diagramas produzidos para esta arquitetura, gerados em PlantUML, são o diagrama de classes e pacotes (Figura 2) e o diagrama de sequência do fluxo de mineração e análise (Figura 1), cujas fontes estão em `docs/diagramas/classes_pacotes.puml` e `docs/diagramas/sequencia_mineracao_metricas.puml`.

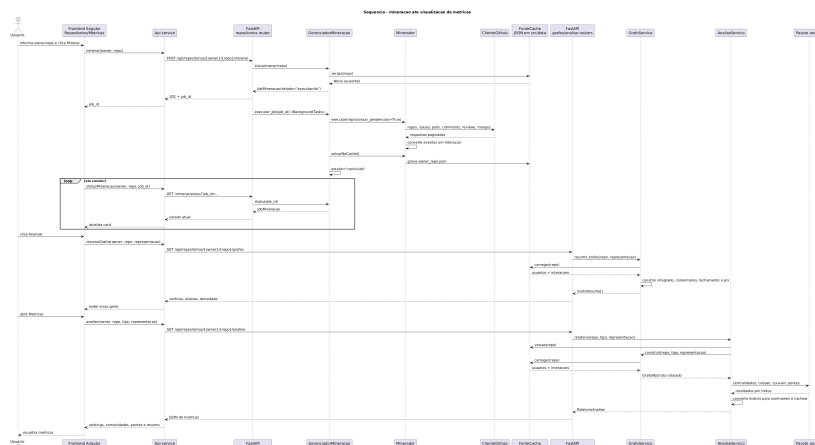


Figura 1. Diagrama de sequência do fluxo de mineração e cálculo de métricas, da ação do usuário no frontend até a visualização dos resultados.

4.1. Estrutura de classes dos grafos

A estrutura de grafos foi modelada com **herança e abstração**, conforme exigido pelo enunciado, em três classes:

- `GrafoAbstrato` (classe abstrata): define a API comum, os atributos compartilhados (rótulos e pesos de vértices, graus de entrada e saída) e os métodos auxiliares (validação de índices, validação de arestas). Concentra também a lógica de alto nível por meio do padrão *Template Method*: operações como `adicionarAresta` e `removeAresta` implementam aqui as regras invariantes (proibição de laços, idempotência, atualização de graus) e delegam apenas a manipulação física da aresta aos métodos abstratos `_inserirAresta` / `_deletarAresta`.
- `GrafoMatrizAdjacencia` (concreta): implementa a API usando uma **matriz de adjacência**.
- `GrafoListaAdjacencia` (concreta): implementa a API usando **listas de adjacência** (dicionários por vértice).

O diagrama de classes e pacotes (Figura 2) foi produzido em PlantUML no arquivo docs/diagramas/classes.pacotes.puml. Ele inclui os pacotes grafos, minerador, backend, analise e gephi.

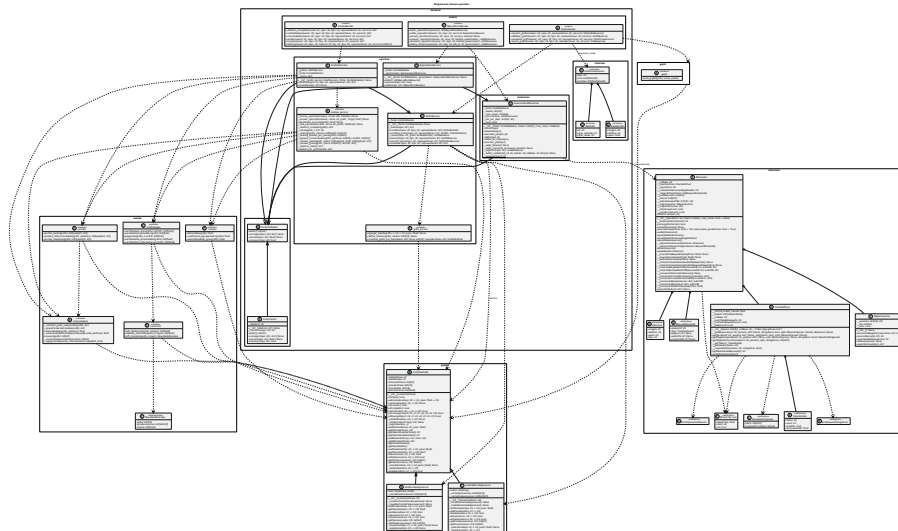


Figura 2. Diagrama de classes e pacotes do backend e do núcleo, incluindo a estrutura de grafos, os serviços, os roteadores e os esquemas de resposta da API.

4.2. API implementada

A estrutura de grafos implementa integralmente uma API que inclui operações de consulta de cardinalidade (quantidade de vértices e arestas), manipulação de arestas (adicionarAresta, removeAresta, existeAresta), relações entre vértices e arestas (isSucessor, isPredecessor, isDivergente, isConvergente, isIncidente), graus (getGrauEntrada, getGrauSaida), pesos e rótulos de vértices e arestas, e propriedades globais (isConexo, isVazio, isCompleto). Além dos métodos obrigatórios, a estrutura disponibiliza operações auxiliares não exigidas pela especificação, como getSucessores e getPredecessores (listas de vizinhos por sentido) e getRotuloVertice / setRotuloVertice (rótulos de vértices), utilizadas internamente pelas métricas e pela serialização dos resultados.

As restrições do enunciado são garantidas pela classe abstrata: o grafo é **simples** (laços e arestas múltiplas são rejeitados), a inserção de arestas é **idempotente** (inserir a mesma aresta duas vezes não a duplica nem altera os graus) e **exceções** são lançadas para índices inválidos (IndexError) e operações inconsistentes (ValueError), como consultar uma relação entre arestas inexistentes.

Decisão de projeto: nomenclatura da API. A API obrigatória do enunciado é especificada com nomes em inglês (por exemplo, getVertexCount, hasEdge, addEdge). Optamos por nomear os métodos em **português** (getQuantidadeVertices, existeAresta, adicionarAresta etc.), mantendo a equivalência semântica um-para-um com a especificação. A decisão visa à consistência com o restante do código-base, integralmente escrito em português. A Tabela 3 apresenta o mapeamento entre os nomes do enunciado e os nomes adotados.

Tabela 3. Mapeamento da API: nomes do enunciado → nomes adotados.

Enunciado (inglês)	Implementado (português)
<code>getVertexCount</code>	<code>getQuantidadeVertices</code>
<code>getEdgeCount</code>	<code>getQuantidadeArestas</code>
<code>hasEdge</code>	<code>existeAresta</code>
<code>addEdge</code>	<code>adicionarAresta</code>
<code>removeEdge</code>	<code>removeAresta</code>
<code>isSucessor</code>	<code>isSucessor</code>
<code>isPredessor</code>	<code>isPredecessor</code>
<code>isDivergent</code>	<code>isDivergente</code>
<code>isConvergent</code>	<code>isConvergente</code>
<code>isIncident</code>	<code>isIncidente</code>
<code>getVertexInDegree</code>	<code>getGrauEntrada</code>
<code>getVertexOutDegree</code>	<code>getGrauSaida</code>
<code>setVertexWeight</code>	<code>setPesoVertice</code>
<code>getVertexWeight</code>	<code>getPesoVertice</code>
<code>setEdgeWeight</code>	<code>setPesoAresta</code>
<code>getEdgeWeight</code>	<code>getPesoAresta</code>
<code>isConnected</code>	<code>isConexo</code>
<code>isEmptyGraph</code>	<code>isVazio</code>
<code>isCompleteGraph</code>	<code>isCompleto</code>

4.3. Exportação para o GEPHI

A exportação para o GEPHI foi implementada em um **módulo dedicado** (`gephi.para_gephi`), em vez de como um método da classe de grafo. A decisão segue o princípio de **separação de responsabilidades**: a classe de grafo concentra-se na estrutura e nas operações de grafo, enquanto a lógica de serialização e escrita em disco fica isolada em seu próprio módulo. O grafo é exportado como um arquivo CSV de arestas, um dos formatos aceitos pelo GEPHI, com as colunas `Source`, `Target`, `Weight` e `Type` (`Directed`), usando os rótulos (*username*s) dos vértices. Além de métricas, rankings, comunidades e pontes, o frontend também renderiza o grafo de forma interativa com a biblioteca `vis.js` (*vis-network*), consumindo uma rota de visualização que devolve os nós e as arestas em JSON. Essa biblioteca é usada apenas na camada visual: ela consome os dados do grafo preparados pelo backend, construídos a partir da biblioteca de grafos implementada pela equipe, e realiza somente a renderização interativa. A exportação em CSV permanece disponível para a análise espacial complementar no GEPHI.

5. Estratégia de Coleta de Dados (Mineração)

O pacote `minerador` é responsável por extrair as interações do repositório por meio da API REST do GitHub. As interações coletadas e transformadas em arestas são:

- **Comentário em *issue***: aresta do autor do comentário para o autor da *issue*;
- **Comentário em *pull request***: aresta do autor do comentário para o autor do *PR*;
- **Fechamento de *issue***: aresta de quem fechou para o autor da *issue* (somente quando são pessoas distintas);
- **Revisão de *pull request***: aresta do revisor para o autor do *PR*;

- **Merge de pull request:** aresta de quem integrou para o autor do *PR*.

Em todos os casos, **auto-interações são descartadas** (o usuário agindo sobre o próprio artefato não gera aresta) e interações repetidas entre o mesmo par e do mesmo tipo têm seus pesos **acumulados**.

5.1. Robustez da coleta

A coleta foi projetada para lidar com as limitações práticas da API do GitHub:

- **Múltiplos tokens:** a mineração suporta uma lista de *tokens* de acesso, contornando o limite de requisições por *token* (*rate limit*).
- **Paginação:** respostas longas (comentários, revisões) são percorridas por paginação.
- **Reprocessamento de pendências:** requisições que falham são registradas como pendências e reprocessadas, garantindo a integridade da coleta (a contabilização ocorre de forma “tudo ou nada” por página, evitando pesos duplicados).
- **Cache em disco:** o resultado da mineração é persistido em JSON (`src/data/<owner>_<repo>.json`), permitindo reanálises sem recoletar os dados.

O enunciado exige testes unitários que garantam a mineração correta das informações; tais testes são descritos na Seção 7.

Na aplicação atual, a mineração pode ser disparada pelo frontend por meio das rotas `POST /api/repositorios/<owner>/<repo>/minerar` e `POST /api/repositorios/<owner>/<repo>/atualizar`. O backend responde imediatamente com um `JobMineracao` e executa o trabalho em *background*. O frontend consulta periodicamente a rota de status até que o job seja concluído ou falhe. O fluxo completo, desde a ação do usuário até a visualização das métricas calculadas, está documentado no diagrama de sequência da Figura 1.

6. Algoritmos e Métricas Implementados

Todas as métricas foram implementadas do zero, utilizando apenas a biblioteca padrão de Python (por exemplo, `heapq` para a fila de prioridade do Dijkstra). O pacote `analise` centraliza, em `caminhos.py`, os utilitários de caminhos mínimos (BFS, Dijkstra e a travessia de Brandes) reaproveitados pelas métricas de distância.

6.1. Métricas de centralidade

- **Grau** (`centralidade_grau`): mede o número de conexões diretas de cada colaborador, separando grau de entrada (interações recebidas) e de saída (realizadas), normalizado por $n - 1$.
- **Proximidade** (`centralidade_proximidade`): mede quão “perto” um colaborador está dos demais, com base em caminhos mínimos (BFS) e normalização de Wasserman–Faust para tratar a desconexão.
- **Intermediação** (`centralidade_intermediacao`): identifica os colaboradores que atuam como *pontes*, calculada pelo **algoritmo de Brandes** (acumulação de dependências).

- **PageRank** (`pagerank`): mede influência considerando a importância de quem está conectado ao colaborador, via iteração de potência com fator de amortecimento $d = 0,85$ e tratamento de nós sem saída (*dangling nodes*).
- **Autovetor** (`centralidade_autovetor`): variante espectral da influência, também por iteração de potência.

6.2. Métricas de estrutura e coesão

- **Densidade** (`densidade`): razão entre o número de arestas existentes e o máximo possível ($E/(V(V - 1))$ para grafo direcionado simples).
- **Coeficiente de agrupamento** (`coeficiente_agrupamento`): tendência de os colaboradores formarem grupos densamente conectados, calculado sobre o grafo subjacente.
- **Assortatividade** (`assortatividade_grau`): correlação de Pearson entre os graus das pontas de cada aresta (fórmula de Newman), indicando se colaboradores muito conectados tendem a interagir entre si.

6.3. Métricas de comunidade

- **Modularidade** (`modularidade`): função de avaliação que mede a qualidade de uma partição em comunidades.
- **Deteção de comunidades** (`louvain`): algoritmo de Louvain, uma heurística gulosa que busca maximizar a modularidade sobre o grafo subjacente ponderado, identificando os grupos de colaboração.
- **Pontes / *bridging ties***: três leituras complementares: `pontes_locais` (arestas cujos extremos não têm vizinhos em comum), `arestas_intercomunidade` (arestas entre comunidades distintas, a partir do resultado do Louvain) e `pontes_classicas` (arestas-ponte cuja remoção desconecta o grafo).

6.4. Serviço de análise

Para uso integrado pela aplicação, o `AnaliseServico` do backend orquestra as funções do pacote `analise` e devolve um relatório consolidado em formato serializável. Esse serviço constrói o grafo solicitado por meio do `GrafoServico`, executa as métricas, converte os índices internos dos vértices para `usernames`, monta rankings ordenados e cacheia o resultado em memória por (`repo`, `tipo`, `representacao`, `versao_do_json`). Assim, uma nova mineração altera o `mtime` do JSON e invalida automaticamente o cache de análise.

7. Estratégia de Testes

A qualidade do código é assegurada por uma suíte de **testes unitários** automatizados (`pytest`), abrangendo os quatro pacotes-núcleo. Cada métrica é validada contra *grafos de referência* pequenos, cujos valores esperados foram calculados manualmente:

- **Grafo estrela, grafo completo, dois triângulos ligados por uma ponte e grafo simétrico** servem de *fixtures* compartilhadas, cada uma com propriedades conhecidas (por exemplo, no grafo completo a densidade é 1; no par de triângulos, o Louvain deve encontrar exatamente duas comunidades).

- A estrutura de grafos é testada de forma **parametrizada** sobre as duas implementações (matriz e lista), garantindo que ambas se comportam de modo idêntico e cobrindo a lógica compartilhada da classe abstrata.
- A mineração é testada com um **cliente falso** (*fake*) que substitui o acesso à rede, validando o processamento de cada tipo de interação, o tratamento de usuários removidos, a acumulação de pesos e o reprocessamento de pendências, atendendo à exigência do enunciado de garantir a mineração correta das informações sem erro.

No estado atual, a suíte conta com **281 testes automatizados** cobrindo estrutura de grafos, mineração, métricas, exportação GEPHI e backend.

8. Resultados

O *snapshot* minerado de `wezterm/wezterm` contém **4.462 usuários** e **9.920 interações distintas**. A Tabela 4 mostra a distribuição das interações agregadas por tipo.

Tabela 4. Interações mineradas no snapshot de `wezterm/wezterm`.

Tipo de interação	Arestas distintas	Peso total
Comentário em issue	7.799	32.646
Comentário em pull request	277	2.896
Fechamento de issue	1.103	1.670
Merge de pull request	398	3.580
Revisão de pull request	343	3.280
Total	9.920	44.072

A Tabela 5 resume os quatro grafos construídos a partir do cache. Como esperado em redes de colaboração de grande escala, os grafos são esparsos: mesmo o grafo integrado, com milhares de colaboradores, possui densidade próxima de zero. Isso indica que a colaboração ocorre em grupos e pares específicos, não como uma rede quase completa.

Tabela 5. Resumo dos grafos construídos para `wezterm/wezterm`.

Grafo	Vértices	Arestas	Densidade
Integrado	4.462	7.972	0,00040050
Comentários	4.449	7.873	0,00039784
Fechamento	1.063	1.103	0,00097705
Pull Requests	521	584	0,00215562

Observa-se que o grafo de comentários concentra a maior parte dos usuários e arestas, o que sugere que a discussão em *issues* e *pull requests* é o principal canal de interação observado pela mineração. O grafo de *pull requests*, embora menor, tende a representar interações tecnicamente mais intensas, pois inclui revisões e *merges*. A aplicação permite explorar essas diferenças selecionando o tipo de grafo e comparando rankings de centralidade, comunidades e pontes no frontend.

A Tabela 6 apresenta as principais métricas de coesão e estrutura do grafo integrado. O coeficiente de agrupamento indica formação de agrupamentos locais, enquanto a assortatividade negativa sugere interação entre colaboradores com graus diferentes. O Louvain identificou 171 comunidades com modularidade 0,3418.

Tabela 6. Métricas de coesão e estrutura do grafo integrado.

Métrica	Valor
Densidade	0,00040
Coefficiente de agrupamento	0,2830
Assortatividade de grau	-0,2358
Modularidade (Louvain)	0,3418
Nº de comunidades	171

A distribuição das comunidades e as cinco maiores comunidades aparecem na Tabela 7. As comunidades somam 4.462 membros; as maiores concentram parte relevante dos colaboradores e incluem mantenedores, bots e usuários recorrentes.

Tabela 7. Resumo das comunidades detectadas pelo Louvain.

Grupo	Valor	Detalhe
200+ membros	3	Distribuição por faixa de tamanho
51 a 200 membros	14	Distribuição por faixa de tamanho
6 a 50 membros	28	Distribuição por faixa de tamanho
2 a 5 membros	126	Distribuição por faixa de tamanho
Isoladas (1)	0	Distribuição por faixa de tamanho
Comunidade 1	1.028	wez, jsqf, imsnif, corbob
Comunidade 2	817	github-actions[bot], MelvDouc, kkanden, TraceBivens
Comunidade 3	478	bew, ipkiss42, spazm, mjec
Comunidade 4	163	Universal-Invariant, diegorocchia, RoastBeefer00, cmvanb
Comunidade 5	128	theEndBeta, aliaksandr-trush, ccaneke, TraceLD

As pontes também foram contabilizadas no grafo integrado: foram encontradas **3.036 pontes locais**, **2.190 pontes clássicas** e **3.117 arestas intercomunidade**. Esses valores reforçam que parte expressiva da rede atua conectando grupos distintos de colaboração.

A Tabela 8 resume os dez primeiros colocados em cada centralidade calculada sobre o grafo integrado. Os valores evidenciam a centralidade de *wez* em diferentes leituras, além da presença de bots e de colaboradores recorrentes em posições de destaque.

Tabela 8. Top 10 de centralidades no grafo integrado.

#	Entrada (ent/sai)	V.	Saída (ent/sai)	V.	PageRank	V.
1	wez (179/1846)	0,0401	wez (179/1846)	0,4138	wez	0,0199
2	halfelf (45/0)	0,0101	github-actions[bot] (1/1629)	0,3652	prabirshrestha	0,0071
3	crabddancing (35/2)	0,0078	bew (25/465)	0,1042	hgaiser	0,0056
4	ShadowS0ng (30/0)	0,0067	loops (10/42)	0,0094	halfelf	0,0043
5	zw963 (28/3)	0,0063	MuhammedZakir (6/36)	0,0081	crabddancing	0,0035
6	prabirshrestha (27/6)	0,0061	no-response[bot] (0/32)	0,0072	ShadowS0ng	0,0034
7	nasyxx (26/2)	0,0058	ghost (19/29)	0,0065	onedr0p	0,0032
8	ModProg (26/8)	0,0058	SuperSandro2000 (15/27)	0,0061	jmbaur	0,0031
9	stevenxxiu (26/3)	0,0058	gegoune (6/27)	0,0061	grant0417	0,0030
10	bew (25/465)	0,0056	valpackett (18/20)	0,0045	JoyceBabu	0,0029
#	Autovetor	V.	Proximidade	V.	Intermediação	V.
1	wez	0,2059	wez	0,4340	wez	0,0977
2	prabirshrestha	0,0738	github-actions[bot]	0,4020	bew	0,0183
3	ModProg	0,0573	bew	0,2816	prabirshrestha	0,0122
4	stevenxxiu	0,0549	MuhammedZakir	0,2544	github-actions[bot]	0,0087
5	hgaiser	0,0514	kadkdaka	0,2543	proski	0,0078
6	bew	0,0514	eugenesvk	0,2535	hgaiser	0,0076
7	joexue	0,0503	mborejdo	0,2534	alexherbo2	0,0055
8	ghost	0,0492	tmccombs	0,2532	poetaman	0,0053
9	crabddancing	0,0491	happenslol	0,2528	xiangpeng2008	0,0042
10	JoyceBabu	0,0490	follower	0,2526	JoyceBabu	0,0039

A Tabela 9 lista as arestas de maior peso, isto é, os pares com maior intensidade acumulada de colaboração no grafo integrado.

9. Telas da Aplicação

Esta seção reúne as telas produzidas pela aplicação web, ilustrando o fluxo de uso da ferramenta, da seleção do repositório à exploração das métricas e à visualização interativa do grafo. As capturas correspondem ao repositório *wezterm/wezterm* analisado na Seção 8.

Tabela 9. Arestas mais pesadas do grafo integrado.

#	Origem → destino	Peso	#	Origem → destino	Peso
1	wez → bew	673	6	wez → jalil-salame	247
2	wez → Danielkonge	340	7	wez → Funami580	236
3	wez → ghost	322	8	wez → mgpinf	231
4	wez → erf	308	9	wez → davidrios	221
5	wez → joexue	252	10	wez → valpacket	212

9.1. Seleção de repositórios e visão geral

A Figura 3 mostra a tela inicial: o usuário informa um repositório para minerar e acompanha os já minerados, cada um com a respectiva data de cache. A Figura 4 apresenta a visão geral dos quatro grafos do repositório selecionado, com o resumo do grafo integrado e, em cada *card*, os botões de métricas, de exportação para o GEPHI e o botão *Visualizar*.

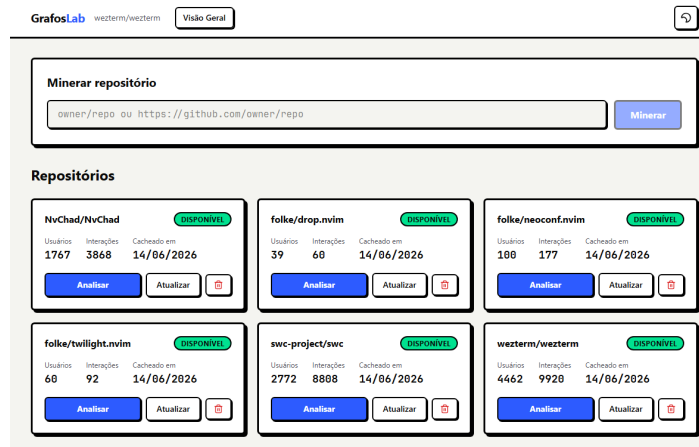


Figura 3. Tela de repositórios: mineração de um novo repositório e listagem dos já minerados, com a data de cache de cada um.

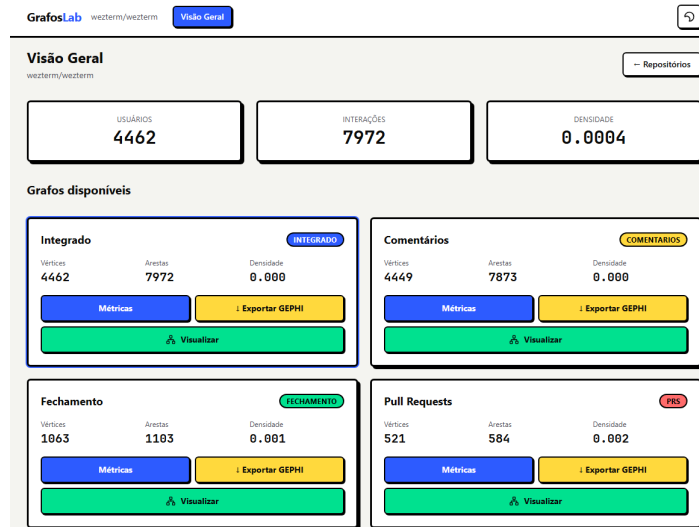


Figura 4. Visão geral dos quatro grafos do repositório, com o resumo do grafo integrado e o botão *Visualizar* em cada *card*.

9.2. Visualização interativa do grafo

A Figura 5 mostra a visualização interativa do grafo em um modal, renderizada no frontend com a biblioteca `vis.js` (*vis-network*) a partir dos nós e arestas devolvidos pela rota de visualização da API.

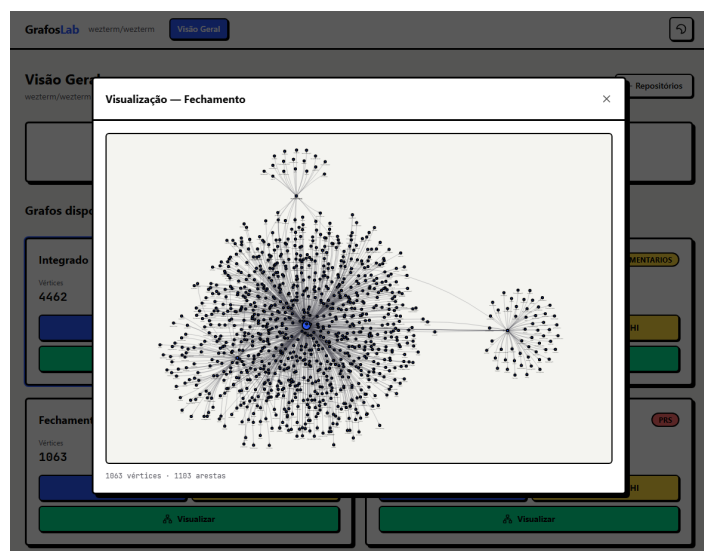


Figura 5. Visualização interativa do grafo no frontend, renderizada com a biblioteca `vis.js` a partir dos dados devolvidos pela API.

9.3. Métricas, comunidades e pontes

A tela de métricas consolida os resultados da análise. A Figura 6 traz o resumo da rede (densidade, coeficiente de agrupamento, assortatividade e modularidade); e a Figura 7 apresenta o ranking de centralidade, com a métrica selecionável e o número de posições exibidas. A tela também possui uma tabela com o ranking das arestas por peso, destacando os pares de colaboradores com interação mais intensa. A detecção de comunidades aparece na Figura 8, com um *card* por comunidade e a modularidade da partição. Abaixo também na tela de métricas, existem 3 tabelas mostrando as pontes locais, clássicas e arestas intercomunidade.

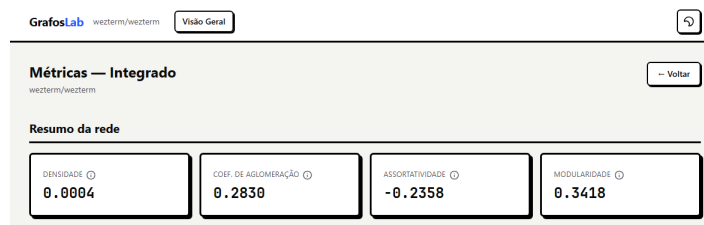


Figura 6. Resumo da rede na tela de métricas: densidade, coeficiente de agrupamento, assortatividade e modularidade.

Métrica: Grau (entrada) — Quantidade de interações recebidas por usuário (normalizada).				
Grau (entrada) Grau (saída) PageRank Autovetor Proximidade Intermediação				
Exibir top: 10 25 50				
#	Usuário	Grau ent.	Grau sai.	Valor
1	wez	179	1846	0.0481
2	halfelf	45	0	0.0101
3	crabdancing	35	2	0.0078
4	Shadow50ng	30	0	0.0067
5	zw963	28	3	0.0063
6	prabirshrestha	27	6	0.0061
7	nasyxx	26	2	0.0058
8	ModProg	26	8	0.0058
9	stevenoxiu	26	3	0.0058
10	bew	25	465	0.0056

Figura 7. Ranking de centralidade, com a métrica selecionável (aqui, grau de entrada) e o número de posições exibidas.



Figura 8. Comunidades detectadas pelo algoritmo de Louvain, com a modularidade da partição e os membros de cada comunidade.

10. Responsabilidades dos Membros

As responsabilidades foram divididas por pacote, com colaboração entre os membros nas etapas de integração, ajustes finais e testes. A Tabela 10 resume a divisão principal.

Tabela 10. Responsabilidades dos membros do grupo.

Membro	Responsabilidades
Vicenzo Fonseca de Mello Souza	Refatoração do minerador; implementação de GrafoAbstrato; implementação de GrafoMatrizAdjacencia em conjunto com Renato; implementação de comunidades.py e pontes.py; testes das partes implementadas.
Renato Douglas Nascimento Silva de Oliveira	Implementação de GrafoMatrizAdjacencia em conjunto com Vicenzo; implementação de partes de centralidade.py em conjunto com Kayke; implementação principal do backend FastAPI; testes das partes implementadas.
Ana Luiza de Freitas Rodrigues	Implementação de GrafoListaAdjacencia; implementação de coesao.py; implementação principal do frontend Angular em conjunto com Kayke; testes das partes implementadas.
Kayke Emanuel de Souza Santos	Implementação da exportação para GEPHI; implementação de caminhos.py; implementação de partes de centralidade.py em conjunto com Renato; implementação principal do frontend Angular em conjunto com Ana Luiza; testes das partes implementadas.
Grupo todo	Implementação inicial do minerador; ajustes finais no backend e no frontend; integração entre os pacotes; revisão geral da aplicação.

11. Conclusão

Este trabalho apresentou uma ferramenta completa para modelar e analisar a rede de colaboração de um repositório *open source*, desde a mineração das interações até a aplicação de métricas de redes complexas, com toda a estrutura de grafos e os algoritmos implementados do zero. A modelagem como grafo direcionado e ponderado, aliada às métricas de centralidade, coesão e comunidade, permite caracterizar quantitativa e qualitativamente a dinâmica de colaboração do projeto analisado. As decisões de modelagem e desenvolvimento foram registradas ao longo do texto, e a corretude da implementação é sustentada por uma ampla suíte de testes automatizados.